

Пакет java.net

Перейдем к рассмотрению средств *Java* для работы с сетью.

Классы, работающие с сетевыми протоколами, располагаются в пакете `java.net`, и простейшим из них является *класс* `URL`. С его помощью можно сконструировать *uniform resource locator (URL)*, который имеет следующий формат:

```
protocol://host:port/resource
```

Здесь `protocol` – название протокола, используемого для связи; `host` – *IP-адрес*, или *DNS-имя* сервера, к которому производится обращение; `port` – *номер порта* сервера (если *порт* не указан, то используется *значение* по умолчанию для указанного протокола); `resource` – имя запрашиваемого ресурса, причем, оно может быть составным, например:

```
ftp://myserver.ru/pub/docs/Java/JavaCourse.txt
```

Затем можно воспользоваться методом `openStream()`, который возвращает `InputStream`, что позволяет считать содержимое ресурса. Например, следующая *программа* при помощи `LineNumberReader` считывает первую страницу сайта `http://www.ru` и выводит ее на *консоль*.

```
import java.io.*;
import java.net.*;

public class Net {
    public static void main(String args[]) {
        try {
            URL url = new URL("http://www.ru");
            LineNumberReader r =
                new LineNumberReader(new
                    InputStreamReader(url.openStream()));
            String s = r.readLine();
            while (s!=null) {
                System.out.println(s);
                s = r.readLine();
            }
            System.out.println(r.getLineNumber());
            r.close();
        } catch (MalformedURLException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Из примера мы видим, что работа с сетью, как и работа с потоками, требует дополнительной работы с исключительными ситуациями.

Ошибка `MalformedURLException` появляется в случае, если строка с `URL` содержит ошибки.

Более функциональным классом является `URLConnection`, который можно получить с помощью метода класса `URLConnection()`. У этого класса есть два метода – `getInputStream()` (именно с его помощью работает `URL.openStream()`)

и `getOutputStream()`, который можно использовать для передачи данных на *сервер*, если он поддерживает такую операцию (многие публичные web-серверы закрыты для таких действий).

Класс `URLConnection` является абстрактным. Виртуальная машина предоставляет реализации этого класса для каждого протокола, например, в том же пакете `java.net` определен класс `HttpURLConnection`. Понятно, что классы `URL` и `URLConnection` предоставляют возможность работы через *сеть* на прикладном уровне с помощью высокоуровневых протоколов.

Пакет `java.net` также предоставляет *доступ* к протоколам более низкого уровня – *TCP* и *UDP*. Для этого сначала надо ознакомиться с классом `InetAddress`, который является *Internet*-адресом, или *IP*. Экземпляры этого класса создаются не с помощью конструкторов, а с помощью статических методов:

```
InetAddress getLocalHost()
InetAddress getByName(String name)
InetAddress[] getAllByName(String name)
```

Первый метод возвращает *IP-адрес* машины, на которой исполняется *Java- программа*. Второй метод возвращает *адрес* сервера, чье имя передается в качестве параметра. Это может быть как *DNS*-имя, так и числовой *IP*, записанный в виде текста, например, "67.11.12.101". Наконец, третий метод определяет все *IP*-адреса указанного сервера.

Для работы с *TCP*-протоколом используются классы `Socket` и `ServerSocket`. Первым создается `ServerSocket` – *сокет* на стороне сервера. Его простейший *конструктор* имеет только один *параметр* – *номер порта*, на котором будут приниматься входящие запросы. После создания вызывается метод `accept()`, который приостанавливает выполнение программы и ожидает, пока какой-нибудь клиент не иницирует соединение. В этом случае работа сервера возобновляется, а метод возвращает экземпляр класса `Socket` для взаимодействия с клиентом:

```
try {
    ServerSocket ss = new ServerSocket(3456);
    Socket client=ss.accept();
    // Метод не возвращает
    // управление, пока не подключится клиент
} catch (IOException e) {
    e.printStackTrace();
}
```

Клиент для подключения к серверу также использует класс `Socket`. Его простейший *конструктор* принимает два параметра – *адрес* сервера (в виде строки, или экземпляра `InetAddress`) и *номер порта*. Если *сервер* принял *запрос*, то *сокет* конструируется успешно и далее можно воспользоваться методами `getInputStream()` или `getOutputStream()`.

```
try {
    Socket s = new Socket("localhost", 3456);
    InputStream is = s.getInputStream();
    is.read();
} catch (UnknownHostException e) {
    e.printStackTrace();
} catch (IOException e) {
```

```
    e.printStackTrace();
}
```

Обратите внимание на обработку исключительной ситуации `UnknownHostException`, которая будет генерироваться, если *виртуальная машина* с помощью операционной системы не сможет распознать указанный *адрес* сервера в случае, если он задан строкой. Если же он задан экземпляром `InetAddress`, то эту ошибку надо обрабатывать при вызове статических методов данного класса.

На стороне сервера *класс* `Socket` используется точно таким же образом – через методы `getInputStream()` и `getOutputStream()`. Приведем более полный пример:

```
import java.io.*;
import java.net.*;
public class Server {
    public static void main(String args[]) {
        try {
            ServerSocket ss = new ServerSocket(3456);
            System.out.println("Waiting...");
            Socket client=ss.accept();
            System.out.println("Connected");
            client.getOutputStream().write(10);
            client.close();
            ss.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Сервер по запросу клиента отправляет число 10 и завершает работу. Обратите внимание, что при завершении вызываются методы `close()` для открытых сокетов.

Класс клиента:

```
import java.io.*;
import java.net.*;
public class Client {
    public static void main(String args[]) {
        try {
            Socket s = new Socket("localhost", 3456);
            InputStream is = s.getInputStream();
            System.out.println("Read: "+is.read());
            s.close();
        } catch (UnknownHostException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

После запуска сервера, а затем клиента, можно увидеть результат – полученное число, 10, после чего обе программы закроются.

Рассмотрим эти классы более подробно. Во-первых, *класс* `ServerSocket` имеет *конструктор*, в который передается, кроме номера порта, еще и *адрес* машины. Это может показаться странным, ведь *сервер* открывается на

той же машине, где работает *программа*, зачем специально указывать ее *адрес*? Однако, если *компьютер* имеет несколько сетевых интерфейсов (*сетевых карточек*), то он имеет и несколько сетевых адресов. С помощью такого детализированного конструктора можно указать, по какому именно адресу ожидать подключения. Это должен быть именно локальный *адрес* машины, иначе возникнет ошибка.

Аналогично, *класс* Socket имеет расширенный *конструктор* для указания как локального адреса, с которого будет устанавливаться соединение, так и локального порта (иначе *операционная система* выделяет произвольный свободный *порт*).

Во-вторых, можно воспользоваться методом setSoTimeout(int timeout) класса ServerSocket, чтобы указать время в миллисекундах, на протяжении которого нужно ожидать подключение клиента. Это позволяет серверу не "зависать", если никто не пытается начать с ним работать. Тайм-аут задается в миллисекундах, нулевое значение означает бесконечное время ожидания.

Важно подчеркнуть, что после установления соединения с клиентом *сервер* выходит из метода accept(), то есть перестает быть готовым принимать новые запросы. Однако, как правило, желательно, чтобы *сервер* мог работать с несколькими клиентами одновременно. Для этого необходимо при подключении очередного пользователя создавать новый *поток* исполнения, который будет обслуживать его, а основной *поток* снова войдет в метод accept(). Приведем пример такого решения:

```
import java.io.*;
import java.net.*;

public class NetServer {
    public static final int PORT = 2500;
    private static final int TIME_SEND_SLEEP = 100;
    private static final int COUNT_TO_SEND = 10;
    private ServerSocket servSocket;

    public static void main(String[] args) {
        NetServer server = new NetServer();
        server.go();
    }

    public NetServer() {
        try{
            servSocket = new ServerSocket(PORT);
        } catch(IOException e) {
            System.err.println("Unable to open Server Socket : " +
e.toString());
        }
    }

    public void go() {
        // Класс-поток для работы с
        //подключившимся клиентом
        class Listener implements Runnable {
            Socket socket;
            public Listener(Socket aSocket) {
                socket = aSocket;
            }
            public void run() {
                try {
                    System.out.println("Listener started");
```

```

        int count = 0;
        OutputStream out = socket.getOutputStream();
        OutputStreamWriter writer = new
            OutputStreamWriter(out);
        PrintWriter pWriter = new PrintWriter(writer);
        while (count<COUNT_TO_SEND) {
            count++;
            pWriter.print(((count>1)?",":"")+ "Say" + count);
            sleeps(TIME_SEND_SLEEP);
        }
        pWriter.close();
    } catch(IOException e) {
        System.err.println("Exception : " + e.toString());
    }
}

// Основной поток, циклически выполняющий метод accept()
System.out.println("Server started");
while (true) {
    try {
        Socket socket = servSocket.accept();
        Listener listener = new Listener(socket);
        Thread thread = new Thread(listener);
        thread.start();
    } catch(IOException e) {
        System.err.println("IOException : " + e.toString());
    }
}

public void sleeps(long time) {
    try {
        Thread.sleep(time);
    } catch(InterruptedException e) {
    }
}
}

```

Пример 2.

Теперь объявим клиента. Эта *программа* будет запускать несколько потоков, каждый из которых независимо подключается к серверу, считывает его ответ и выводит на *консоль*.

```

import java.io.*;
import java.net.*;

public class NetClient implements Runnable {
    public static final int PORT = 2500;
    public static final String HOST = "localhost";
    public static final int CLIENTS_COUNT = 5;
    public static final int READ_BUFFER_SIZE = 10;

    private String name = null;

    public static void main(String[] args) {
        String name = "name";
        for (int i=1; i<=CLIENTS_COUNT; i++) {
            NetClient client = new NetClient(name+i);
            Thread thread = new Thread(client);
            thread.start();
        }
    }
}

```

```

    }

    public NetClient(String name) {
        this.name = name;
    }

    public void run() {
        char[] readed = new char[READ_BUFFER_SIZE];
        StringBuffer strBuff = new StringBuffer();
        try {
            Socket socket = new Socket(HOST, PORT);
            InputStream in = socket.getInputStream();
            InputStreamReader reader = new InputStreamReader(in);
            while (true) {
                int count = reader.read(readed, 0,
                    READ_BUFFER_SIZE);
                if (count==-1) break;
                strBuff.append(readed, 0, count);
                Thread.yield();
            }
        } catch (UnknownHostException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
        System.out.println("client " + name + " read : " + strBuff.toString());
    }
}

```

Пример 3.

Теперь рассмотрим UDP. Для работы с этим протоколом и на стороне клиента, и на стороне сервера используется *класс* `DatagramSocket`. У него есть следующие конструкторы:

```

DatagramSocket()
DatagramSocket(int port)
DatagramSocket(int port, InetAddress laddr)

```

При вызове первого конструктора *сокет* открывается на произвольном доступном порту, что уместно для клиента. *Конструктор* с одним параметром, задающим *порт*, как правило, применяется на серверах, чтобы клиенты знали, на каком порту им нужно пытаться устанавливать соединение. Наконец, последний *конструктор* необходим для машин, у которых присутствует несколько сетевых интерфейсов.

После открытия сокетов начинается обмен *датаграммами*. Они представляются экземплярами класса `DatagramPacket`. При отсылке сообщения применяется следующий *конструктор*:

```

DatagramPacket(byte[] buf, int length,
                InetAddress address, int port)

```

Массив содержит данные для отправки (созданный пакет будет иметь длину, равную *length*), а *адрес* и *порт* указывают получателя пакета. После этого вызывается метод `send()` класса `DatagramSocket`.

```

try {
    DatagramSocket s = new DatagramSocket();

```

```

byte data[]={1, 2, 3};
InetAddress addr =
    InetAddress.getByName("localhost");
DatagramPacket p =
    new DatagramPacket(data, 3, addr, 3456);
s.send(p);
System.out.println("Datagram sent");
s.close();
} catch (SocketException e) {
    e.printStackTrace();
} catch (UnknownHostException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
}
}

```

Для получения *датаграммы* также создается экземпляр класса `DatagramPacket`, но в *конструктор* передается лишь *массив*, в который будут записаны полученные данные (также указывается ожидаемая *длина* пакета). *Сокет* необходимо создать с указанием порта, иначе, скорее всего, сообщение просто не дойдет до адресата. Используется метод `receive()` класса `DatagramSocket` (аналогично методу `ServerSocket.accept()`, этот метод также прерывает выполнение потока, пока не придет *запрос* от клиента). Пример реализации получателя:

```

try {
    DatagramSocket s =
        new DatagramSocket(3456);
    byte data[]=new byte[3];
    DatagramPacket p =
        new DatagramPacket(data, 3);
    System.out.println("Waiting...");
    s.receive(p);
    System.out.println("Datagram received: "+
        data[0]+" ", "+data[1]+" ", "+data[2]);
    s.close();
} catch (SocketException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
}
}

```

Если запустить сначала получателя, а затем отправителя, то можно увидеть, что первый напечатает содержимое полученной *датаграммы*, а потом программы завершат свою работу.

В заключение приведем пример сервера, который получает *датаграммы* и отправляет их обратно, дописав к ним *слово* `received`.

```

import java.io.*;
import java.net.*;

public class DatagramDemoServer {
    public static final int PORT = 2000;
    private static final int LENGTH_RECEIVE = 1;
    private static final byte[] answer = ("received").getBytes();
    private DatagramSocket servSocket = null;
    private boolean keepRunning = true;
    public static void main(String[] args) {
        DatagramDemoServer server = new DatagramDemoServer();
        server.service();
    }
}

```

```

    }

    public DatagramDemoServer() {
        try {
            servSocket = new DatagramSocket(PORT);
        } catch (SocketException e) {
            System.err.println("Unable to open socket : " + e.toString());
        }
    }

    protected void service() {
        DatagramPacket datagram;
        InetAddress clientAddr;
        int clientPort;
        byte[] data;
        while (keepRunning) {
            try {
                data = new byte[LENGTH_RECEIVE];
                datagram = new DatagramPacket(data, data.length);
                servSocket.receive(datagram);
                clientAddr = datagram.getAddress();
                clientPort = datagram.getPort();
                data = getSendData(datagram.getData());
                datagram = new DatagramPacket(data, data.length,
                    clientAddr, clientPort);
                servSocket.send(datagram);
            } catch (IOException e) {
                System.err.println("I/O Exception : " + e.toString());
            }
        }
    }

    protected byte[] getSendData(byte b[]) {
        byte[] result = new byte[b.length+answer.length];
        System.arraycopy(b, 0, result, 0, b.length);
        System.arraycopy(answer, 0, result, b.length, answer.length);
        return result;
    }
}

```